Gestão Fácil: colinha das pastas do código

Pasta com.gestaofacil, na ordem em que vale a pena falar. A linha em laranja é a frase para dizer.

Caminho de um pedido: a tela chama o **controller**, que chama o **service**, que usa o **repository** para ler ou gravar no **model** (banco de dados).

0. GestaoFacilApplication (arquivo solto)

Falar: "Este é o botão de ligar do sistema. Quando damos Run, o Spring Boot começa por aqui."

1. model: o que o sistema guarda

Falar: "Cada classe aqui vira uma tabela no banco."

- **Empresa:** as empresas clientes, cada uma com seus dados separados
- **Usuario:** quem acessa; o **Perfil** define Administrador, Motorista ou Operador
- **Veiculo:** os veículos; **StatusVeiculo** (disponível ou em uso) e **TipoCombustivel**
- **CentroCusto:** o local ou obra ligado ao veículo
- **UsoVeiculo:** cada saída e devolução (a classe mais importante)
- **AnexoUsuario:** documentos anexados ao funcionário

2. repository: como busca no banco

Falar: "Aqui ficam as consultas, sem escrever SQL: o Spring monta pelo nome do método. Tem um repositório por tabela."

- VeiculoRepository, UsuarioRepository, EmpresaRepository, CentroCustoRepository, UsoVeiculoRepository, AnexoUsuarioRepository

Frase forte: toda consulta leva a empresa junto (ex.: findByIdAndEmpresaId). É isso que impede uma empresa de ver dados de outra.

3. service: as regras do sistema (a pasta mais importante)

Falar: "É aqui que ficam as regras de negócio."

- **UsoVeiculoService:** registra saída e devolução; confere tudo primeiro e só depois grava
- **VeiculoService:** muda o status do veículo sozinho (em uso / disponível)
- **UsuarioService, EmpresaService, CentroCustoService:** regras de cada cadastro
- **ValidadorDeCpf:** confere se o CPF é válido
- **GeradorDeSenhaTemporaria:** cria a senha temporária do funcionário novo
- **ControleDeTentativasService:** conta erros de senha e bloqueia a conta após 5
- **AnexoUsuarioService e ArmazenamentoDeAnexos:** guardam anexos fora do projeto, com nome aleatório
- **LancamentoRecusadoException e AnexoInvalidoException:** mensagens de erro prontas para o usuário
- **UnidadesFederativas e FormatoDeVeiculo:** listas e formatos de apoio

4. controller: a ligação com a tela

Falar: "Recebe o clique ou o formulário, chama o service e devolve a tela certa."

- Um por assunto: **VeiculoController, UsuarioController, EmpresaController, CentroCustoController**
- **UsoVeiculoController:** saída e devolução do motorista; **VeiculoEmUsoController:** painel do administrador
- **LoginController, TrocaSenhaController, InicioController, AnexoUsuarioController**
- **Subpasta form** (SaidaForm, VeiculoForm...): recebem o que foi digitado; o formulário nunca mexe direto na tabela

5. security: login e proteção

Falar: "Cuida de quem pode entrar e do que cada um pode fazer."

- **AutenticacaoPorEmpresaProvider:** login levando em conta a empresa
- **AutenticacaoSucessoHandler e AutenticacaoFalhaHandler:** o que acontece quando o login acerta ou erra
- **SenhaTemporariaInterceptor:** obriga a trocar a senha temporária no primeiro acesso
- **UsuarioAutenticado:** guarda quem está logado e de qual empresa
- Demais classes: apoio ao login por empresa e ao aviso de tentativas

6. config: configurações gerais

Falar: "Configurações que valem para o sistema inteiro."

- **SecurityConfig:** quais telas cada perfil acessa
- **WebMvcConfig:** configurações das telas
- **CargaInicial:** cria os dados iniciais (empresa e usuário de teste)

Se perguntarem de uma classe que você não conhece: abra o arquivo e leia os nomes dos métodos, que costumam explicar o que ela faz. Se não der, diga: "essa parte o José explica melhor, ele acompanhou essa implementação".